# Training a sub-4B SLM for multi-turn security patching

A minimal, end-to-end pipeline: take a small instruction-tuned model, teach it to **fix security vulnerabilities** in real repositories through multi-turn tool use, and close the loop **SFT** → **RL** → **eval**. The emphasis here is on the methodology and the behavioral trajectory

**Student model:** Qwen3-1.7B (sub-4B, native `<tool_call>` chat template, already function-calling capable).

- **Benchmark:** PatchEval-Verified — 230 real CVEs (Go / JavaScript / Python), each shipped as a Docker image with the vulnerable repo, a proof-of-concept exploit, and a functional test suite.
- **Teacher (for SFT data only):** a large frontier model (Gpt-5.6-terra) driving the exact same tool loop.
- **Pipeline:** teacher trajectory collection → LoRA SFT (behavior cloning) → GRPO (on-policy RL with a shaped reward) → held-out eval.

## Results at a glance

All three stages evaluated on the **same 30 held-out CVEs** (10 Go / 10 JS / 10 Python), under the identical fixed environment, graded on the strict criterion (exploit blocked **and** functional tests pass). SFT is behavior-cloned on 32 verified teacher trajectories spanning 14 CVEs (Go 11, Python 9, JavaScript 12); GRPO runs on top of it for 14 steps (stopped at a 3h45m cap). Numbers are totals across the 30 episodes; rollouts are sampled at temperature 1.0, so read the **directional** shifts, not the last digit.

| Metric | Baseline (base 1.7B) | SFT (teacher traces) | GRPO (from SFT) |
|---|---|---|---|
| **Resolved** | **0 / 30** | **0 / 30** | **0 / 30** |
| `bash` calls (exploration) | 348 | 62 | 74 |
| `view_file` calls | 47 | 54 | **103** |
| `replace_lines` calls (edits tried) | 15 | **133** | 52 |
| edits that landed | 15 | 16 | **19** |
| **land rate** (landed / tried) | — | 12% | **37%** |
| non-empty patches | 9 | 11 | **14** |
| `submit` calls | 14 | 9 | 16 |
| `no_tool_call` (gave up) | 9 | 19 | 12 |
| `max_turns` timeouts | 7 | 2 | 2 |

**Read of the table.** The score is flat at zero — the honest ceiling for a 1.7B model on real CVEs (see §6). What moves is *behavior*, and it moves the right way at every stage. The base model burns turns on exploration (**348** `bash` calls) and times out 7 times. SFT collapses that exploration and makes the model commit: `replace_lines` jumps 15 → 133. GRPO then improves edit **quality, not just quantity**: it *cuts* blind attempts (133 → 52) while **landing more of them** (16 → 19), tripling the land rate (**12%** → **37%**), producing more real patches (11 → 14), grounding more before it edits (`view_file` 54 → 103). This is the opposite of a reward-hacking signature, and it is a direct effect of the gated reward (§5): once empty/no-op edits stopped paying, the policy was pushed toward edits that actually apply. The one thing that never moves is a *correct* fix — even with the improved editing, the 1.7B model hallucinates the exact fix content, so 0 CVEs resolve. RL made the model a better *editor*; it could not make it a better *reasoner*.

## 1. The loop, and that it actually closes

The agent works only through tools inside the container, with the network off:

- `bash` (read-only exploration — grep/find/cat),
- `view_file` (line-numbered file view),
- `replace_lines` (the single edit primitive: replace an inclusive line range with new text),
- `submit` (finish; the submission is the agent's own `git diff`).

One environment, one agent loop, shared by every policy (teacher, base, SFT, RL) so that the only thing that changes between stages is the model's weights. Grading is delegated to PatchEval's own scripts in a fresh container

The loop runs cleanly: teacher traces are collected and graded, the successful ones are cloned via SFT, the SFT checkpoint seeds GRPO, and all three stages (base / SFT / RL) are evaluated on the same held-out split. The RL stage made **real policy-gradient updates every step**

---

## 2. Why this benchmark

PatchEval was chosen because it grades the **real objective**, not a proxy. "Did the patch fix the vulnerability?" is answered by actually running the exploit against the patched code, and "did the patch keep the program working?" by running the project's own test suite. That gives an unambiguous, execution- grounded `1.0` signal — exactly what you want an RL reward anchored to — and it covers three languages so the model can't overfit one ecosystem.

The cost is operational weight: every rollout and every grade needs a container. We run rootless (udocker in user namespaces, no Docker daemon, no sudo), unpack onto a RAM disk for throughput, and isolate the network with an unprivileged namespace. This is the scope we *paid* for in order to keep the reward honest.

---

## 3. Why this model, this data, this algorithm

**Model — Qwen3-1.7B.** Small enough to train and roll out on a single consumer GPU (RTX 2080Ti) inside the time
budget, but already instruction-tuned with a native tool-call format, so no effort is wasted teaching basic syntax. It fits the sub-4B constraint with headroom. Gemma E2B would is actually around 4.2B with 2B active parameters, which would not be possible to train on 2080Ti

**SFT data — teacher trajectories, not an off-the-shelf dataset.** We surveyed the obvious tool-use SFT corpora given first. The problem was fit, not availability:

- most were **shallow** (single-turn or near-single-turn) and don't resemble multi-turn patch work;
- most **don't follow the chat tool-call template** our environment uses, so adopting them meant heavy reformatting that would introduce its own distribution shift;
- and the domains were largely **irrelevant** to editing a real code repo under an exploit.

So the highest-leverage SFT data is trajectories from *our own environment*: run the teacher through the exact tool loop, keep the trajectories that actually fix the CVE (verified by the grader), and clone those. The data is on-distribution by construction — same tools, same prompt, same observation format — and every example is a known-good demonstration of the target behavior.

**RL algorithm — Ideal choice would be to do GRPO + on-policy distillation using open-source large LLM (500B+ params) but since that's not feasible and computationally very expensive, I chose GRPO.** It needs no separate value/critic network (cheaper on a small GPU), it group-normalizes rewards within a set of rollouts for the same CVE (so advantage is "better than my other attempts on this exact problem," which is well-suited to sparse success), it naturally helps with sparse rewards because of exploration and it's on-policy, so it reinforces the behavior the current model actually produces. A frozen copy of the SFT adapter is the KL reference, keeping the policy from drifting into degenerate text.

**Scope cuts (deliberate).** Short SFT (a small set of verified teacher trajectories — 32 across 14 CVEs, not thousands);

LoRA rather than full fine-tuning; a modest GRPO run (small groups, few CVEs per step) that is *not* expected to converge in the time budget. The goal was a clean, debuggable loop that shows behaviour improvement, not a converged model.

---

## 4. Reward design

The terminal objective is PatchEval's own check, worth **1.0**: the exploit no longer fires **and** the functional tests still pass. That signal is far too sparse for a ~2B model that starts near zero — most rollouts would get exactly `0.0` and provide no gradient. So the reward is **shaped as a sequence of increasing milestones** that give dense, directional credit along the path a real fix takes:

```
read the vulnerable file      → small credit
land a valid edit             → larger credit
edit the actually-vulnerable file → larger credit
submit                        → larger credit
exploit fixed + tests pass    → full 1.0  (always dominates the sum of milestones)
```

Two design choices matter here. First, the milestones are **capped well below 1.0**, so an actual fix is always strictly better than any amount of partial credit — the shaping guides the policy without ever competing with the real objective. Second, the credit is deliberately **weighted toward the decisive actions** (landing an edit, submitting) because those were the behaviors the student under-produced; the reward points at the exact gap we observed.

The ground truth used for milestone credit (which file is vulnerable) is **training-only** and **never shown to the model** and never used at eval — it shapes the reward, it does not leak the answer.

---

## 5. Preventing reward hacking

A shaped reward invites gaming, so we took several layered steps to discourage it rather than relying on penalties after the fact:

- **Guards at the tool level.** We intercept network access and repository deletion in the environment, and the edit tool refuses to touch test/PoC files — so the obvious shortcuts (fetching the fix online, wiping the repo, rewriting the tests through the editor) are closed off before they run.
- **Test changes are stripped in the reward path.** For milestone credit, any changes to test or PoC files are stripped from the diff before crediting, so points can only come from touching real code.
- **Edit/submit credit is gated on a real diff (a hardening step added after we caught the hole).** The first version of the shaped reward paid `valid_edit` for any `"Edited"` tool response and `submitted` for any `submit` call — regardless of what the final diff contained. That is gameable: an empty submit scored the submit credit, and a no-op edit (writing a line's existing text back, which produces no diff) plus a submit scored both, for free, without ever changing the code. We closed this by gating both milestones on the **gradeable diff** (`gradeable_patch()`): credit is paid only when the final patch is non-empty, is not a no-op, and touches at least one real code file (test/docs/config-only diffs count as nothing). After the gate, an empty-or-no-op submit earns **0**, and the only way to collect edit/submit credit is to leave a real code change behind. This single change is what flipped RL's behavior from "submit more junk" to "land more real edits" (see §6).
- **Milestone weights were also softened** (sum of partial credit ≈ 0.4, well under the 1.0 an actual fix earns), so a real fix always dominates any stack of partial credit and the shaping can only *guide*, never *compete with*, the true objective.
- **Functionality is graded, not just the exploit.** The most important anti-hack: the obvious cheat is to *break or delete the vulnerable function* so the exploit can't fire. That would pass the security PoC. We therefore require the **functional test suite to pass as well** (`unit_test.sh`), not just that the attack was prevented (`fix-run.sh`). A patch that neuters the code to dodge the exploit fails functionality and scores zero. "Fixed"

means *the exploit is gone and the program still works.*

Together these close the cheap wins and force whatever credit the policy earns to come from genuinely moving toward a real patch — which is exactly what the post-gate numbers show.

---

## 6. Honest evaluation: what changed, and what didn't

All three stages were evaluated on the same held-out CVEs, under the identical (fixed) environment, with the stricter exploit-plus-functionality grade.

**The base model's failure mode was over-exploration.** Left to itself it spends almost all of its turns running `bash` — grepping and listing files — and very rarely commits an edit. It behaves like it's perpetually investigating and almost never acts.

**So SFT and RL were aimed squarely at one thing: teach the model to edit, to see if better editing could get at least one fix correct.** SFT on the teacher trajectories shifts the behavior exactly as intended — exploration collapses (`bash` 348 → 62) and the model starts committing targeted `replace_lines` edits (15 → 133). GRPO then does something better than "more of the same": it improves edit **quality**. It *cuts* the number of blind edit attempts (133 → 52) while **landing more of them** (16 → 19), tripling the land rate (12% → 37%), producing more real patches (11 → 14), and reading the code more before it acts (`view_file` 54 → 103). This is the concrete, measurable evidence that **the model learned to make edits that actually apply**, and it is a direct consequence of the gated reward (§5): once empty and no-op edits stopped paying, the only way to earn credit was to leave a real change behind, so the policy learned to ground its edits instead of spraying attempts.

**And yet the end result is zero resolved CVEs, at every stage.** This is reported plainly: a 1.7B model is simply too weak for the hardest part of the task. Even when it correctly localizes the region to change, it **hallucinates** — inventing file paths that don't exist, guessing line numbers that don't line up with what it just read, and writing plausible-looking but incorrect fix code. The edits land in the repo; they just aren't *correct* fixes. Fixing a real CVE requires precise, grounded reasoning over exact code spans, and that is beyond this model's capability regardless of how we shape the reward.

The honest summary: **the behavioral trajectory is real and positive (exploration → editing → better, grounded editing), the absolute score is zero, and the gap between the two is model capability, not pipeline correctness.** Regressions and flat scores are expected and fine here; the point was to show the loop works and to read the behavior change it produces.

---

## 7. Caveats worth stating

- **RL did not converge, by design.** The run is short and the reward-per-step is noisy because each step samples only a couple of CVEs; the signal to read is "updates are happening and editing behavior strengthens," not a smooth learning curve.
- **The RL run was stopped at step 14 of 20 by a wall-clock cap**
- **Credit assignment is approximate, a trade-off of fitting Qwen's small context.** To keep long multi-turn rollouts inside the model's limited window, we trim context (keep the last few tool outputs full, stub older ones). We deliberately apply the *same* trimming at train time as at rollout time to keep it as on-policy as possible — but because trimming is relative to the end of the conversation, an early turn's training context is stubbed slightly more than what it actually saw when acting. So it's a mitigated approximation, not a perfectly faithful one; with a larger-context model the trimming (and this gap) would largely disappear.

---

## 8. What I'd do with a week instead of an afternoon

- **Bigger, verified SFT set, multi-turn curated.** Collect many more teacher trajectories, filter hard for the clean "locate → edit → submit" shape, and include a few *failure-then-recovery* trajectories so the student learns to correct a bad edit instead of repeating it.

- **Off-policy distillation in the SFT stage, with an open-source teacher.** Our teacher is a closed API, so the SFT we actually ran is behavior cloning on *sampled text* — the student only sees the teacher's chosen tokens, never the distribution behind them. Given an **open-weight large teacher**, the SFT stage could instead be true off-policy distillation: minimize the KL between the student's next-token distribution and the teacher's **full logits** over the trajectory, so the small model learns to *match the larger model's distribution* rather than merely imitate its samples. That transfers far more signal per token (the teacher's uncertainty and its ranking of the near-miss tokens, not just the argmax), is much more sample-efficient than cloning sampled traces, and typically yields a tighter, better-calibrated student — a strictly stronger initialization to hand to GRPO than the plain SFT used here.
- **A step up in model size.** Move to a 4B (or 7B at the edge of the constraint) student; the qualitative behavior is already right, and the missing piece — correct, grounded edits — is precisely what more capacity buys. Keep the 1.7B as the cheap, honest baseline.
- **Scale and stabilize RL.** Larger groups, more CVEs per step, gradient accumulation, then actually let it run long enough to read a convergence trend rather than just "updates are happening."
- **Report with proper variance.** Multiple eval seeds and pass@k, so the behavioral deltas come with error bars instead of single stochastic runs.
- **Try more interesting GRPO objective.** One of my recent paper https://arxiv.org/abs/2605.09359 introduces bi-level GRPO objective which I think could be greatly suited here. What I can do given more compute and training time is let model make security patches across K rollouts and then compare them across generations and use that as the training signal. This gives more info to model when writing a patch as it gets it's previous generation's edit as info. So the new advantage could be A_intra (how the edit performed across it's batch) and A_inter (how edit improved compared to previous generation).